

Data Detective – Audit 4

**Entwicklungsprojekt WS 25/26
von Florian Roß und Karim Khemiri**

Inhaltsverzeichnis

1. Erneuter Überblick POCs (Florian)
 2. Achievements und Daily Challenges (Florian)
 3. Persönliche Statistiken (Florian)
 4. Extra Schwierigkeitsgrade und neue Methoden (Florian)
 5. Main Prototype Life Präsentation (Karim)
 6. Generelle Improvements (Karim)
 7. Auswertung des Fachgesprächs (Karim)
 8. Derzeitiger Projektstand, Ausblick und Herausforderungen (Karim)
- (Quellen und Kontakt zum Fachgesprächspartner)

Durchgeführte PoCs

PoC 107 Stilistische Manipulationen ✓

Es soll geprüft werden, ob Diagramme über rein visuelle Stilmittel (Fehlende Beschriftungen, Style (Farben, Strichstärken, Hervorhebungen etc.), Overlaps / Occlusion) manipuliert werden können.

- Zwei Neue Manipulationstypen (Balkenverzerrung/Farb-Highlighting)

PoC 109 Reale Anwendungsbeispiele zu Statistik-Manipulationen ✓

Es soll geprüft werden, ob nach Lösungsbildung passende hochwertige Realbeispiele bereitgestellt werden

- Funktioneller Historisches Beispiel Button (+Einbindung NewsScreen)

PoC 110 Gamification ✓

Es soll geprüft werden, ob die App genügend Spaß und Erfolgsergebnisse liefert, zur Gewährleistung der Nutzer Motivation.

- Achievements,
- Daily Challenges,
- 2 neue Spielmodi (Double und Survival),
- PersonalScreen (Stats, Streaks, Mastery Chart)

PoC 112 Persistenz Spiel- und Lernfortschritt ✓

Es soll geprüft werden, ob Nutzerfortschritt zuverlässig gespeichert und geladen werden kann (XP, Level, persönliche Statistik, Achievements)

Restliche Pocs wurden erfüllt:

Poc 107: Zwei neue Stilistische Manipulationen

(Balkenverzerrung/Farbhightlighting)
(folgende Folien mehr infos)

Poc 109: Es wurde ein funktionieller Historisches Beispiel Button hinzugefügt, diese zeigt nach einer gelösten Aufgabe ein passendes Realbeispiel (Gleicher

Manipulationstyp sowie Diagrammtyp) mit Erklärung an, bei einer Double Aufgabe werden zwei Realbeispiele angezeigt
Desweiteren kann man sich alle Realbeispiele auch auf der News Seite anschauen. Es gibt zu jeder (kompatiblen) Manipulationstyp/Diagramm Kombination ein passendes Realbeispiele (Werte dafür noch unvollständig)

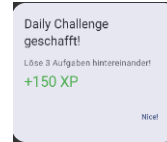
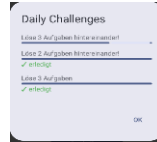
Poc 110: Achievements, Daily Challenges, 2 neue Spielmodi (Double und Survival), Personalscreen(Stats, Streaks, MasteryChart) (folgende Folien mehr infos)

Poc 112: Nutzerfortschritt wird jetzt persistent gespeichert.

Achievements und Daily Challenges

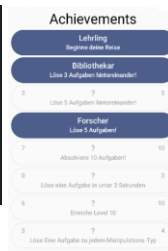
Challenges/
Achievements als
Datenobjekte mit
definierten
Conditions

```
data class DailyChallenge {  
    val id: String,  
    val hint: String,  
    val condition: DailyCondition,  
    val required: Int,  
    val xp: Int,  
}
```



Challenges resettet und rotieren
automatisch täglich

```
val today = LocalDate.now().toString()  
val storedDate = prefs[PreferencesKeys.DAILY_DATE]  
  
if (storedDate != today) {  
    prefs[PreferencesKeys.DAILY_DATE] = today  
    prefs[PreferencesKeys.DAILY_SOLVED] = 0  
    prefs[PreferencesKeys.DAILY_DONE] = 0  
    prefs[PreferencesKeys.DAILY_STREAK] = 0  
    prefs[PreferencesKeys.DAILY_BEST] = 0  
    prefs[PreferencesKeys.DAILY_COMPLETED] = ""  
  
    ManipulationType.entries.forEach {  
        prefs[PreferencesKeys.dailySolvedKey("type" = it)] = 0  
    }  
}
```



Persistente Speicherung des Fortschritts lokal über
DataStore (Nach jeder Aufgabe)

```
if (correctAnswer) { Bsp: Gesamt gelöste Aufgabenanzahl  
    // Stats  
    val solved = (prefs[PreferencesKeys.SOLVED_QUESTIONS] ?: 0) + 1  
    prefs[PreferencesKeys.SOLVED_QUESTIONS] = solved  
}
```

Daily Challenge Pop-Up bei Erfüllung

```
todayChallenges.forEach { challenge ->  
    val progress = challenge.condition.currentValue(profile)  
    if (progress >= challenge.required &&  
        challenge.id !in completedSet) {  
        completedSet = completedSet + challenge.id  
        popup = challenge  
    }  
}
```

Achievements und Daily Challenges
gleiche Logik (Definiert in Data classes,
conditions in enum classes)
Conditions erfüllt, wenn getrackte Werte
bestimmtes Ziel erreichen, welches
Conditions definieren
Persistente Speicherung(im userProfile,
mit PreferencesKeys über DataStore) der
einzelnen getrackten
Werte(HighestStreak, Questions

solved/done, Level, AllManipulationsSolved)
sowie der Werte für die Daily challenges.

Daily Challenges sowie deren Condition-Werte werden täglich zurückgesetzt (Datum wird gespeichert und mit aktuellem verglichen, um neuen Tag zu erkennen)

Bei Erreichen einer Daily Challenge ploppt ein Pop-Up auf, welches anzeigt, dass man die Challenge geschafft hat und wie viel XP man bekommt.

Im Daily Challenges Fenster werden die Aufgaben und der aktuelle Fortschritt angezeigt.

Im Achievements Screen werden die Achievements + Fortschritt aufgelistet, unlocken eines Achievements schaltet einen auswählbaren Titel frei

Persönliche Statistiken

Speicherung der allgemeinen Statistiken über die PreferencesKeys die auch für die Achievements genutzt werden

Snapshot alle 10 Aufgaben für Entwicklungsvergleich

```
// Snapshot alle 10 Aufgaben
if (done % 10 == 0) {
    prefs[PreferencesKeys.SNAP_XP] = prefs[PreferencesKeys.XP] ? 0 : 0
    prefs[PreferencesKeys.SNAP_LEVEL] = prefs[PreferencesKeys.LEVEL] ? 1 : 1
    prefs[PreferencesKeys.SNAP_SOLVED] = prefs[PreferencesKeys.SOLVED_QUESTIONS] ? 0 : 0
    prefs[PreferencesKeys.SNAP_WRONG] = prefs[PreferencesKeys.WRONG_QUESTIONS] ? 0 : 0
    prefs[PreferencesKeys.SNAP_HIGHEST_STREAK] = prefs[PreferencesKeys.HIGHEST_STREAK] ? 0 : 0
    ManipulationType.entries.forEach { type =>
        prefs[PreferencesKeys.snapshotKey(type)] = prefs[PreferencesKeys.solvedKey(type)] ? 0 : 0
    }
    ManipulationType.entries.forEach { type =>
        prefs[PreferencesKeys.snapshotKey(type)] = prefs[PreferencesKeys.failedKey(type)] ? 0 : 0
    }
}
```

Mastery Graph

```
// Mastery Graph
val masteryPath = Path()
types.forEachIndexed { index, type ->
    val value = mastery[type].coerceIn(0f, 1f) ? 0f : 0f
    val angle = angleStep * index - Math.PI.toFloat() / 2
    val r = radius * value
    val x = center.x + r * cos(angle)
    val y = center.y + r * sin(angle)
    if (index == 0) masteryPath.moveTo(x, y)
    else masteryPath.lineTo(x, y)
}
masteryPath.close()
```

Hilfsfunktion, die Richtig-Anzahl pro Manipulation trackt

```
fun masteryManipulation(
    solved: Map<ManipulationType, Int>,
    failed: Map<ManipulationType, Int>
): Map<ManipulationType, Float> {
    return ManipulationType.entries.associateWith { type ->
        val s = solved[type] ? 0 : 0
        val f = failed[type] ? 0 : 0
        val total = s + f
        if (total == 0) 0f else s.toFloat() / total
    }
```

Streaks pro Manipulation

```
//Zähler für jeweiligen Manipulationstypen
manipulationType?.let { type ->
    val key =
        if (correctAnswer)
            PreferencesKeys.solvedKey(type)
        else
            PreferencesKeys.failedKey(type)
    prefs[key] = (prefs[key] ? 0) + 1
    //Streaks für einzelne Manipulationstypen erhöhen
    manipulationType?.let { type ->
        val streakKey = PreferencesKeys.streakKey(type)
        val bestKey = PreferencesKeys.bestStreakKey(type)
        if (correctAnswer) {
            val newStreak = (prefs[streakKey] ? 0) + 1
            prefs[streakKey] = newStreak
            val best = prefs[bestKey] ? 0 : 0
            if (newStreak > best) prefs[bestKey] = newStreak
        } else {
            prefs[streakKey] = 0
        }
    }
}
```

Die getrackten Werte für die Achievement Conditions werden auch für den Personalscreen verwendet. Zudem wird noch die highest Streak des Survival Modes getrackt.

Desweiteren wird getrackt, wieviel man richtig/falsch pro Manipulationstyp hat(über Maps), mit diesen Daten kann dann ein Manipulation Mastery Graph

erzeugt werden, welcher die Stärken/Schwächen pro Typ anzeigt (es wird ein Durchschnittswert zwischen 0-1 berechnet über Hilfsfunktion masterByManipulation) Außerdem wird alle 10 Aufgaben ein Snapshot erstellt, um den User einen vorher/nachher vergleich zu seinen Leistungen zu bieten, diese werden normal wie auch die anderen Statistiken gespeichert, aber nur alle 10 Aufgaben aktualisiert und dann unter den normalen Werten in grün angezeigt sowie also orangener MasteryChart

Zuletzt werden noch Streaks für die einzelnen Manipulationstypen getrackt (current/best) (basiert auf Expertenfeedbackgespräch)

Extra Schwierigkeitsgrade und neue Methoden

Balkenverzerrung:
Berechnung des Intensitätswert in
ManipulatedChart

```
val targetDistortion =  
    if (!showCorrect && distortion != null)  
        distortion.intensity  
    else  
        1f
```

Verzerrung in Chart Composable mit zuvor
erechnetem distortionFactor

```
//Verzerrung der Balkenhöhe bei DISTORTED_BAR_LENGTH  
val relativeHeight = normalized.pow(x = distortionFactor)  
val barHeight = relativeHeight * chartHeight  
val yTop = paddingTop + chartHeight - barHeight
```

Farb-Highlighting:

Alle bis auf ein zufälliger Balken/Punkt
bekommen eine leichte Transparenz

```
if (!highlightEnabled || highlightedIndex == null) {  
    return baseColor  
}  
return if (index == highlightedIndex) {  
    baseColor  
} else {  
    baseColor.copy(alpha = 0.85f) //leicht transparent  
}
```

**Double und
Survival Modi**

```
enum class ManipulationMode {  
    9 Uses  
    SINGLE,  
    6 Uses  
    DOUBLE,  
    10 Uses  
    SURVIVAL  
}
```

Manipulationen im Task werden
nun als Liste gespeichert, damit
Taskgenerator auch Double-
aufgaben generieren kann

```
val manipulations: List<Manipulation>
```

Survival Modus

```
//Survivalmode logik  
if (manipulationMode == ManipulationMode.SURVIVAL) {  
    if (!isCorrect) {  
        survivalStreak++  
        if (survivalStreak > survivalBest) {  
            survivalBest = survivalStreak  
            // Persistieren in Database  
            viewModelScope.launch {  
                userData.saveSurvivalBest(survivalBest)  
            }  
        }  
    } else {  
        survivalRunResult = SurvivalResult(  
            streak = survivalStreak,  
            best = survivalBest  
        )  
        survivalStreak = 0  
        return  
    }  
}
```

Run beendet

Aufgaben geschafft

2

Bester Run

6

Zurück zum Menü

Zwei Neue Manipulationsmethoden:
Balkenverzerrung: Die Balkenlänge wird
mit zufälliger Intensität verzerrt (die
Balken-Beschriftungen/Werte sowie
Achsenwerte bleiben aber korrekt)
Farb Highlighting: Alle bis auf ein zufälliger
Balken bekommen leichte Transparenz,
somit sticht ein balken heraus, bei
Linechart ähnlich jedoch bleiben hier zwei
Abschnitte(also 3 Punkte und 2 Lines)

gehighlighted während der Rest leicht transparent wird.

3 Spielmodi: SINGLE, DOUBLE, SURVIVAL

Single: normaler Spielmodus mit einer Manipulation pro Diagramm,

Double: zwei kombinierte (kompatible) Manipulationen pro Diagramm,

Um Double Aufgaben zu stellen, wurde die Speicherart der Manipulationen für Tasks auf List geändert und der Taskgenerator, Manipulatedchart und Viewmodel entsprechend darauf angepasst, ManipulatedChart guckt hierfür, ob mehrere Manipulationswerte herausextrahiert werden müssen und übergibt dann mehrere Werte .

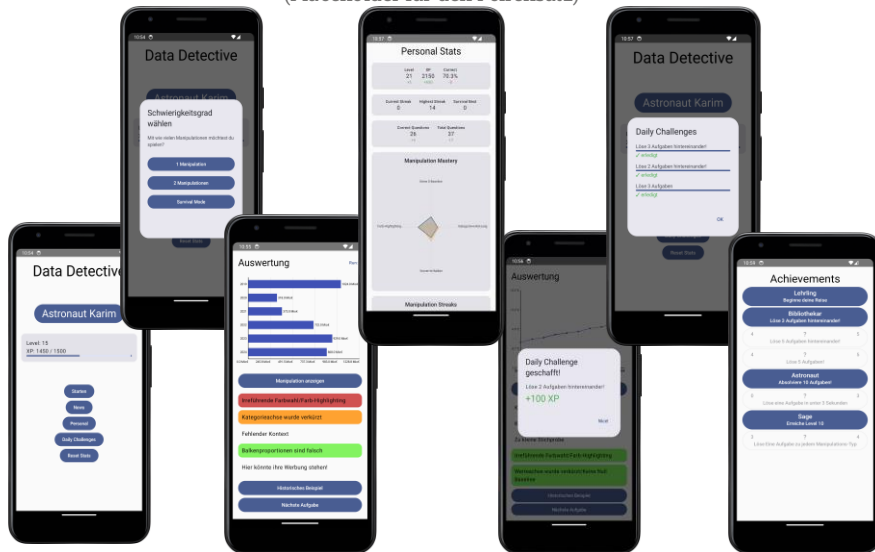
Survival (basiert auf

Expertenfeedbackgespräch): Endless Run

Mode bis man eine Aufgabe falsch hat. Die ersten 3 Aufgaben immer single, danach zufällig ob single oder double, SurvivalStreak wird getrackt und am Ende des Runs angezeigt

Data Detective Rapid Prototype

(Placeholder für den Foliensatz)



Generelle Improvements

Bugfixes:

- Statistiken und dessen Animationen glitchen nicht mehr beim Wechsel der Screens
- Alle Screens nun korrekt an Homescreen gebunden
- Alle Fade ins und Fade outs funktionieren nun einwandfrei

UI:

- Konsistentere und kompaktere Screen-Darstellungen
- Run Counter als Info für den Nutzer

Allgemein:

- Nutzer muss nun die richtige Aufgabenauswahl auswählen und wird über Schwierigkeitsgrad hingewiesen
- Run Counter zählt beendete Aufgaben der Session
- Persistente Speicherung aller Nutzerdaten
- Zurücksetzen der Nutzerdaten (Mit Schutz)
- Nutzer wird über Daily Challenges und erlangte XP durch Pop-Up hingewiesen
- "News"-Tab würde überarbeitet und werden nun genutzt als Real-Beispiele innerhalb der Aufgaben

Im Rahmen des letzten Projektsprints wurden nicht nur viele Funktionalitäten hinzugefügt auf die Fokus gelegt wurde, sondern es wurden auch viele generelle Improvements und kritische Bugfixes berücksichtigt. Dazu gehören sowohl Quality of Life Features innerhalb der UI/UX als auch sinnvolle Änderungen der Navigation und der auszuführenden Logik. Es sind unseres Wissens nach auch derzeit

keine weiteren Bugs bekannt die wir nicht beheben konnten. Besonders die persistente Speicherung aller Daten ist für unsere App mit all den verschiedenen Nutzerdaten und dessen Snapshot Logik gut umgesetzt worden und leicht skalierbar.

Gamification/MCI-Fachgespräch mit Simon Ludwig Schulte

Reines visuelles Feedback und Progress tracking unzureichend, Belohnungen ohne klare Sinnhaftigkeit eher wertlos

Der Personal Screen:

- Top Scores und besonders Streak Counter als starke Motivationsquelle
- Durch Zahlen und eigener Statistik vielseitig ansprechend

Historische Beispiele:

- Dienen als gutes Tool zur Entwicklung von Rahmenbedingungen für zusätzliches Interesse intrinsischer Motivation
- Binden Interesse und emotionales Investment an Kompetenz und Lernerfolg

Achievements:

- laut Feedback immer noch zu wenig Kompetenz/Skill-basiert
- Benötigen stärkere Belohnungen, Ziele oder andere Sinnhaftigkeit statt reines Completionist-Prinzip und Titel

Daily Challenges und Level System:

- Visuelles reines Progress-Tracking keine ausreichende Motivationsquelle, besitzt im Gegensatz zu Achievements keine wirkliche Belohnung, ausbesserbar durch Kauf-, Tausch-Mechaniken oder Customization

-> **Empfehlungen:** Noch stärkerer Fokus auf Streaks (umgesetzt), XP/Level Ausbau, Survival Mode als Spielmodus orientiert an Streaks (umgesetzt), Freischaltbarer Content, RAMP- als Gamification Guideline, Verknüpfung der einzelnen Systeme

Das Fachgespräch mit Simon Schulte (wissenschaftlicher Mitarbeiter für Social Computing der TH und unser Experte für Gamification) hat sich als besonders hilfreich herausgestellt. Er hat uns klar vermitteln wo Stärken und wo Schwächen innerhalb unseres Systems zu finden sind und hat uns im Gespräch helfen können Lösungsansätze für Probleme direkt zu evaluieren. Der Personal Screen bekam

innerhalb des Gesprächs viel Lob während sich unser Level System und unsere Daily Challenges als größte derzeitige Schwachstelle herausstellten. Weitere Details zum Gespräch sind im neuen Artefakt für Audit 4 einsehbar. Simon gab uns außerdem Empfehlungen für den weiteren Verlauf und das RAMP-Prinzip als Guideline für zukünftige Anpassungen des Gamification Konzepts. Durch unsere gute Vorarbeit und Code Qualität waren der Challenge Mode und die individuellen Streaks für jede Manipulationsmethode innerhalb eines Tages umsetzbar und haben damit noch in das Audit 4 gepasst.

Derzeitiger Projektstand, Ausblick und Herausforderungen

Stand:

- Zielerreichungsgrad: Hoch, Konzept wurde voll in vertikalem Prototyp umgesetzt
- Deutliche Ausarbeitung der Grundlegenden Funktionen und der Gamification
- Massiver Sprung in Code Qualität von Audit 3 zu Audit 4
- Bestätigung das genutzte Bibliotheken und die Kompetenz ausreichen um Projekt weiter zu führen

Ausblick:

- Viele Möglichkeiten zum Ausbau: Mehr Schwierigkeitsgrade, Kategorien, (Online-Mode)
- Verbesserte Gamification auf Basis des Fachgesprächs
- Fokus für Audit 4 war Gamification deshalb zukünftig eher wieder auf Methoden als Fokus zur Implementierung
- Nutzertests

Herausforderungen:

- Passende Reabeispiele finden schwieriger als erwartet
- Ob alle Aufgabentypen / Manipulationsmethoden im Rahmen der App relevanten Lernerfolg bieten unschlüssig
- "Künstlich" angepasste Statistiken weiterhin ein Problem, sehr Methodenabhängig

Da wir in der Lage waren viele unserer Ziele bis Audit 4 auch umzusetzen ist der derzeitige Stand unseres Projekts ausgereift. Unser Main Prototyp beschreibt die vollständige Vertikale User-Experience auch wenn einige der Sub-Systeme (besonders die Gamification) noch deutliche Verbesserungen ermöglichen oder auch benötigen um die App tatsächlich für Nutzer anzubieten.

Dennoch bietet der Main Prototype eine gute Basis dafür. Es hat sich außerdem gezeigt das finden von passenden Realbeispielen deutlich schwieriger ausfällt als erwartet, da die Datensätze den eine gewisse Qualität erfüllen müssen um als Realbeispiel zu fungieren. Außerdem benötigt das zeigen von Realbeispielen innerhalb der Aufgaben sehr spezifische Statistiken. Durch unseren Fokus der Vertikalität unseres Prototypen und der Gamification sind außerdem derzeit erst 4 Statistikmethoden umgesetzt, daher lassen sich diese nicht alle auf ihre Nutzbarkeit validieren auch wenn klar für uns ist, dass diese technisch schnell im weiteren Ausblick des Projekts umgesetzt werden könnten. Trotz des erfolgreichen Fachgesprächs sollten sich zukünftige Nutzertests dennoch als sehr hilfreich erweisen.

Quelle zum RAMP-Prinzip und Verweis zum Fachgesprächspartner

https://www.th-koeln.de/personen/simon_ludwig.schulte/

<https://www.gamified.uk/gamification-framework/the-intrinsic-motivation-ramp/>